

Vue.js 반응형 레이아웃 & 차트 시각화 등

실무 프로젝트로 배우는 모던 프론트엔드 대시보드 개발



학습 목표

반응형 레이아웃 설계 (CSS Grid, Bootstrap)

Chart.js + vue-chartjs 데이터 시각화

Vuetify 활용 다크모드 & 전문 UI



기술 스택

Vue 3

Vite

CSS Grid

Bootstrap

Chart.js

Vuetify



최종 결과물

실습 기반의 금융 대시보드 UI를 완성합니다.



전체 프로젝트 코드 제공

Table of Contents

01

반응형 레이아웃 & 그리드 시스템

CSS GRID

BOOTSTRAP

02

차트 시각화

CHART.JS

VUE-CHARTJS

03

Vuetify UI & 다크모드

VUETIFY 3

THEMING

04

전체 프로젝트 코드

통합 소스 코드 리뷰

05

마무리 & 다음 단계

확장 가이드

CHAPTER

01

반응형 레이아웃 설계와 그리드 시스템

CSS Grid와 Bootstrap을 활용하여
다양한 화면 크기에 대응하는
유연한 UI 구조를 설계합니다.



학습 목표

Learning Objectives

반응형 레이아웃 개념 이해

유동적인 너비와 브레이크포인트(Breakpoint)의 원리

CSS Grid로 카드형 레이아웃 만들기

display: grid와 미디어쿼리 활용 실습

Bootstrap Grid 시스템 활용

클래스 기반으로 빠르게 반응형 화면 구성하기

실제 대시보드 만들 때 가장 먼저 부딪히는 게 레이아웃입니다.
오늘은 두 가지 핵심 기술로 직접 카드형 UI를 만들어봅니다.

왜 반응형 레이아웃이 필요한가?

사용자 환경의 다양성

단일 해상도 기준의 개발은 불가능합니다.

사용자는 다양한 디바이스로 금융 서비스를 접속합니다.

Mobile

360px ~

Tablet

768px ~

Laptop

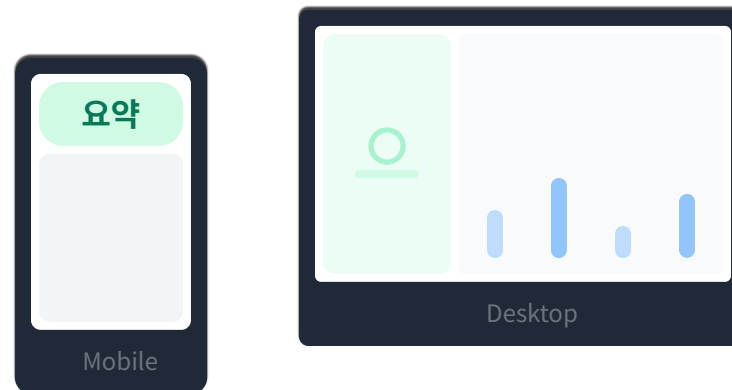
1024px ~

Wide

1920px ~

핀테크 서비스 사례

USE CASE



모바일: 핵심 지표 요약 PC: 차트, 테이블, 필터 등 복합 UI

One Code, Any Screen

코드 한 번 작성으로
모든 해상도를 커버하는 효율성

반응형 레이아웃의 핵심 요소

1. 유동적인 너비 (Fluid Width)

고정 픽셀(px) 대신 화면 크기에 따라 변하는 상대 단위를 사용합니다.

% (Percentage)

fr (Fraction)

vw / vh

66%

2. 브레이크포인트 (Breakpoint)

레이아웃이 변경되는 기준 너비 지점입니다. 디바이스 종류에 대응합니다.

600px

1024px

3. 미디어쿼리 (Media Query)

CSS에서 화면 조건에 따라 다른 스타일을 적용하는 기술입니다.

```
@media (max-width: 600px) {  
  .container {  
    flex-direction: column;  
  }  
}
```

4. 레이아웃 패턴

화면이 좁아질 때 배치가 어떻게 변할지 결정합니다.



Desktop



Mobile

CSS Grid 소개 및 기본 문법

2차원 레이아웃

Grid는 행(Row)과 열(Column)을 동시에 제어하는 2차원 시스템입니다.

핵심 속성

display: grid
grid-template-columns
gap

오늘의 목표

카드 8개를 그리드로 배치하고, 화면 크기에 따라 4열 → 2열
→ 1열로 자동 변경되는 반응형 구조 만들기

```
style.css

1 .grid-container {
2   /* 이 컨테이너를 그리드 레이아웃으로 선언 */
3   display: grid;
4   /* 가로 공간을 4등분 (1fr * 4) */
5   grid-template-columns: repeat(4, 1fr);
6   /* 행(Row)의 높이 설정 (선택사항) */
7   grid-template-rows: auto;
8   /* 아이템 사이의 간격 16px */
9   gap: 16px;
10  padding: 20px;
11 }
```

fr Fraction (비율 단위)

repeat() 반복 함수

미디어쿼리로 반응형 만들기

브레이크포인트

화면 너비(Width)가 특정 지점에 도달했을 때 레이아웃을 변경하는 기준점입니다.

반응형 규칙

PC	기본값	4열
Tablet	$\leq 1024\text{px}$	2열
Mobile	$\leq 600\text{px}$	1열

Key Point

미디어쿼리 안에서 변경할 속성만 덮어쓰기(Override) 하면 됩니다. 나머지는 그대로 상속됩니다.

style.css

```
1 /* 기본 PC 화면 (1024px 초과) */
2 .grid-container {
3   grid-template-columns :repeat(4,1fr);
4 }
5
6 /* 태블릿 (1024px 이하) → 2열로 변경 */
7 @media (max-width :1024px ){
8   .grid-container {
9     grid-template-columns :repeat(2,1fr);
10  }
11 }
12
13 /* 모바일 (600px 이하) → 1열로 변경 */
14 @media (max-width :600px ){
15   .grid-container {
16     grid-template-columns :repeat(1,1fr);
```

ResponsiveGrid.vue 전체 코드

Vue의 역할

템플릿은 단순히 구조만 표현합니다. 복잡한 로직 없이 v-for로 카드만 생성하고, 배치는 전적으로 CSS에 위임합니다.

CSS의 역할

Grid 선언: 카드들을 2차원 그리드로 배치

미디어쿼리: 화면 폭에 따라 열 개수 자동 조절
(4열 → 2열 → 1열)

src/components/ResponsiveGrid.vue

```
1 <template>
2 <divclass="grid-container" >
3 <divclass="card"v-for="n in 8":key="n">
4 CSS Grid 카드 {{n}}
5 </div>
6 </div>
7 </template>

8 <style>
9 .grid-container {
10 display:grid;
11 grid-template-columns :repeat(4,1fr); /* 기본 4열 */
12 gap:16px;
13 padding:20px;
14 }

15 /* 태블릿: 1024px 이하에서 2열로 변경 */
16 @media(max-width:1024px){
17 .grid-container {
```


Bootstrap 프레임워크 개요



Bootstrap이란?

The world's most popular front-end open source toolkit

트위터(Twitter)에서 개발한 오픈소스 UI 프레임워크로, **HTML, CSS, JS**를 기반으로 반응형 웹을 빠르고 쉽게 제작할 수 있도록 도와주는 도구입니다.



역사 (History)

2011년 Mark Otto와 Jacob Thornton이 개발. 현재 v5 버전까지 진화하며 웹 표준을 선도.



컴포넌트 (Components)

버튼, 카드, 네비게이션, 모달 등 미리 디자인된 수많은 UI 요소 제공.



빠른 개발 속도

미리 정의된 클래스로
즉각적인 UI 구축



Mobile First

강력한 12컬럼
반응형 그리드 시스템

장점과 단점

VS

장점 (Pros)

- ✓ 높은 브라우저 호환성
- ✓ 방대한 커뮤니티 & 문서
- ✓ 일관된 디자인 품질

단점 (Cons)

- ✗ 정형화된 디자인 (Bootstrap Look)
- ✗ 불필요한 스타일 로딩 가능성
- ✗ DOM 구조가 복잡해질 수 있음

Bootstrap Breakpoints & Sizes

📱 그리드 시스템 브레이크포인트 (v5 기준)

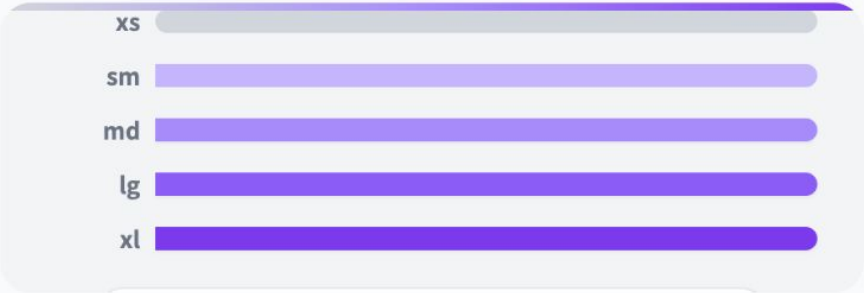
min-width 기준

Prefix	Size Info	Dimensions	Device Example
xs	X-Small	< 576px	📱 모바일 세로 (iPhone, Galaxy)
sm	Small	≥ 576px	📱 모바일 가로 (Large Phones)
md	Medium	≥ 768px	📱 태블릿 (iPad Mini, Galaxy Tab)
lg	Large	≥ 992px	💻 소형 노트북 / 태블릿 가로
xl	X-Large	≥ 1200px	💻 일반 데스크탑 모니터
xxl	XX-Large	≥ 1400px	💻 고해상도 대형 모니터

</> 클래스 사용 패턴

```
<div class="col-12 col-md-6 col-xl-4">  
  
</div>
```

- ✓ **col-12**: 기본(모바일)에서 전체 너비 (1열)
- ✓ **col-md-6**: 768px 이상에서 절반 너비 (2열)
- ✓ **col-xl-4**: 1200px 이상에서 1/3 너비 (3열)



Bootstrap Grid 한눈에 보기

1 INSTALLATION

```
→ project npm install bootstrap
# main.js에서 import 'bootstrap/dist/css/bootstrap.min.css'
import 'bootstrap/dist/js/bootstrap.bundle.min.js'
```

2 USAGE STRUCTURE

```
1<!-- 전체 폭 중앙 정렬 -->
2<div class="container">
3
4<!-- 가로줄 (Row) -->
5<div class="row">
6
7<!-- 12칸 중 4칸 차지 (33%) -->
8<div class="col-4">
9<p>Column 1</p>
10</div>
```

클래스 기반 레이아웃

Class-based System

CSS를 직접 작성하지 않고, HTML 클래스 이름만으로 레이아웃을 제어합니다. 생산성이 매우 높고 팀 프로젝트에서 일관성을 유지하기 쉽습니다.

`.container`

`.row`

`.col-*`

핵심 포인트

12 컬럼 시스템

화면 가로폭을 12등분하여 자유롭게 조합 (3+3+6, 4+4+4 등)

강력한 반응형

col-md-6처럼 브레이크포인트 접두사를 사용해 기기별 레이아웃 지정

CSS Grid vs Bootstrap Grid

CSS Grid



완전한 레이아웃 자유도



2차원 레이아웃 강력함



복잡한 디자인 구현



학습 곡선 존재



직접 CSS 작성 필요

Bootstrap Grid



빠른 개발 속도



팀 협업 용이



일관된 디자인 시스템



12칸 그리드 제한



클래스 의존적

CSS Grid vs Bootstrap Grid



CSS Grid

Custom Layout Design

이럴 때 선택하세요:

디자이너의 시안이 복잡하고 독창적일 때

2차원(가로+세로) 배치를 정밀하게 제어해야 할 때

HTML 구조를 단순하게 유지하고 싶을 때 (시멘틱 마크업)

Learning Curve



Bootstrap Grid

Rapid Prototyping

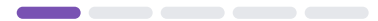
이럴 때 선택하세요:

빠르게 관리자 페이지나 대시보드를 만들 때

여러 개발자가 공통된 규칙으로 협업해야 할 때

12컬럼 기반의 표준 레이아웃으로 충분할 때

Learning Curve



CHAPTER

02

데이터 인사이트를 위한 차트 시각화

Chart.js와 vue-chartjs를 연동하여
데이터를 직관적으로 표현하고
사용자 경험을 향상시키는 방법을 배웁니다.



학습 목표

Learning Objectives

Chart.js + vue-chartjs 설치/연동

Vue 환경에서 차트 라이브러리를 올바르게 세팅하는 방법

바(Bar) 차트 컴포넌트 제작

데이터 바인딩, 옵션 설정, 컴포넌트화 실습

반응형 데이터 갱신 구현

버튼 클릭 등 이벤트에 따른 차트 리렌더링 처리

숫자 테이블보다 차트가 훨씬 강력합니다. 특히 금융 서비스에서 '추세'와 '비중'을 보여주는 시각화는 필수 역량입니다.

Chart.js 설치 및 모듈 등록

패키지 설치

핵심 라이브러리인 chart.js와 Vue 래퍼인 vue-chartjs 두 가지 패키지를 모두 설치해야 합니다.

```
npm install chart.js vue-chartjs
```

모듈 시스템

Tree Shaking: 필요한 모듈만 가져와서 등록하는 방식입니다.

등록 필수: 사용하려는 차트 타입(Bar)이나 기능(Tooltip)을 등록해야만 작동합니다.

Tip

ChartJS.register(...) 코드는 차트 컴포넌트 내부 혹은 main.js 같은 전역 진입점에서 한 번만 호출해도 됩니다.

BarChart.vue (script setup)

```
1 import{
2   Chart as ChartJS,
3   BarElement, // 막대 그래프용 요소
4   CategoryScale, // x축 (카테고리)
5   LinearScale, // y축 (선형 스케일)
6   Tooltip, // 마우스 오버 툴팁
7   Legend // 범례 표시
8 } from 'chart.js'

9 // 필요한 모듈을 Chart.js 코어에 등록
10 ChartJS.register(
11   BarElement,
12   CategoryScale,
13   LinearScale,
14   Tooltip,
15   Legend
16 )

17 // 이제 <Bar /> 컴포넌트가 정상 작동합니다.
```

BarElement 막대 차트 필수

CategoryScale X축 라벨(문자열) 필수

LinearScale Y축 숫자 필수

BarChart.vue 핵심 코드

컴포넌트 활용

<Bar /> 컴포넌트에 데이터를 props로 전달하여 차트를 렌더링합니다.

데이터 반응성

ref()로 감싼 데이터가 변경되면 차트가 자동으로 다시 그려집니다.

버튼 클릭 시 랜덤 데이터를 생성하여 동적 업데이트를 시뮬레이션합니다.

핵심 포인트

Vue의 Reactivity System 덕분에 데이터만 바꾸면 UI(차트)는 알아서 업데이트됩니다.

src/components/BarChart.vue

```
1 <template>
2 <div class="chart-wrapper">
3 <Bar :data="chartData" :options="chartOptions" />
4 <button @click="updateData">데이터 업데이트</button>
5 </div>
6 </template>

7 <script setup>
8 import { ref } from 'vue'
9 import { Bar } from 'vue-chartjs'
10 // ... ChartJS 컴포넌트 등록 로직 생략 ...

11 const chartData = ref({
12   labels: ['1분기', '2분기', '3분기', '4분기'],
13   datasets: [{ label: '매출', data: [10, 20, 30, 25] }]
14 })

15 function updateData() {
16   // 배열의 값을 랜덤하게 변경 → 차트 자동 갱신
17   chartData.value.datasets[0].data = newDataArray
18 }
19 </script>
```


데이터 구조 이해 및 실습 아이디어

chartData 구조

```
const chartData = ref({
  labels: ['1분기', '2분기', '3분기', '4분기'],
  datasets: [
    {
      label: '매출(억 원)',
      backgroundColor: '#4caf50',
      data: [10, 20, 30, 25]
    }
  ]
})
```

주요 옵션 (chartOptions)

responsive: true 부모 컨테이너 크기에 맞춰 자동 리사이즈

maintainAspectRatio: false 높이를 CSS로 자유롭게 지정 가능 (비율 무시)

실습 과제 도전



1. 레이블 변경하기

기존 '분기' 대신 실제 주식 종목명이나 날짜로 X축을 변경해보세요.

```
labels: ['삼성전자', 'SK하이닉스'...]
```



2. 데이터셋 추가 (비교 분석)

'예상 매출'과 '실제 매출' 두 가지 막대 그래프를 나란히 그려보세요.

```
datasets: [ { ... }, { ... } ]
```



3. 사용자 입력 반영

input 태그로 숫자를 입력받아 버튼 클릭 시 차트 데이터를 갱신해보세요.

```
v-model @click="update"
```

CHAPTER

03

Vuetify로 완성하는 전문 UI & 다크모드

Vue 전용 UI 프레임워크인 Vuetify를 활용하여 전문적인 디자인과 기능을 갖춘 대시보드를 구현합니다.



학습 목표

Learning Objectives

Vuetify로 상단 바 및 대시보드 레이아웃 구현

v-app-bar, v-card 등 핵심 컴포넌트 실습

다크모드 토글 기능과 테마 관리

클릭 한 번으로 전환되는 테마 시스템 구축

반응형 그리드 시스템 심화 적용

Vuetify Grid(v-row, v-col)로 유연한 배치

복잡한 UI는 직접 CSS를 짜기보다 잘 만들어진 프레임워크를 쓰는 게 효율적입니다. Vuetify의 강력함을 느껴보세요.

Vuetify란 무엇인가?



Vue.js 전용 UI 프레임워크

Material Design Component Framework

Google의 **Material Design** 가이드라인을 Vue 컴포넌트로 완벽하게 구현한 라이브러리입니다. 디자인 고민 없이 고품질의 UI를 빠르게 구축할 수 있습니다.

KEY FEATURES

방대한 컴포넌트

버튼, 카드, 테이블, 폼 등 80+ UI 요소 제공

테마 시스템

다크모드, 컬러 테마, SASS 커스터마이징

접근성 (A11y)

스크린 리더 지원 및 키보드 내비게이션

반응형 그리드

모바일 우선 12컬럼 레이아웃 시스템

장단점 분석

장점

- ✓ 일관성 있는 전문적 UI 디자인
- ✓ 매우 빠른 프로토타이핑 속도
- ✓ 강력한 커뮤니티와 문서화

단점

- ⚠ 상대적으로 큰 번들 사이즈
- ⚠ 세밀한 커스텀 디자인의 어려움
- ⚠ Material Design 스타일에 종속됨

BEST FOR

언제 사용하면 좋을까?



엔터프라이즈
대시보드



관리자(Admin)
페이지

Vite 프로젝트에 Vuetify 설정

1 NPM INSTALL

```
→ npm install vuetify
→ npm install vite-plugin-vuetify
```

2 MAIN ENTRY (SRC/MAIN.JS)

```
// components, directives: Vuetify 컴포넌트를 전역 등록
// theme: 라이트/다크 테마 정의
const vuetify = createVuetify({
  components,
  directives,
  theme: {
    defaultTheme: 'light',
    themes: {
      light: { dark: false },
      dark: { dark: true }
    }
  }
})
```



Vite Config 설정

vite.config.js

```
import vuetify from 'vite-plugin-vuetify'
export default defineConfig({
  plugins: [
    vue(),
    vuetify({ autoImport: true }) // 자동 импорт
  ]
})
```

vite-plugin-vuetify를 사용하면 컴포넌트를 일일이 import 하지 않아도 필요한 것만 자동으로 불러와 (Tree Shaking) 번들 크기를 최적화합니다.

✓ 설정 체크리스트



테마 시스템 (Theme)

createVuetify 옵션에서 light/dark 테마를 정의합니다. 나중에 theme.global.name 값을 변경하여 다크모드를 전환할 수 있습니다.

Vuetify 대시보드 & 다크모드 구현



기본 구조 컴포넌트

`<v-app>` 모든 Vuetify 애플리케이션의 루트 래퍼

`<v-main>` 상단/측면 바를 제외한 콘텐츠 영역 자동 리사이징

`<v-switch>` 다크모드 전환을 위한 토글 UI



다크모드 로직

Vuetify는 `useTheme` 훅을 통해 전역 테마 상태를 관리합니다.

Tip: `global.name.value`를 변경하면 앱 전체 스타일이 즉시 반영됩니다.

```
VuetifyDashboard.vue

1 <template>
2   <v-app>
3     <!-- 상단 네비게이션 바 -->
4     <v-app-bar color="primary">
5       <v-toolbar-title>Vuetify 대시보드</v-toolbar-title>
6       <v-spacer></v-spacer>
7
8       <!-- 다크 모드 토글 스위치 -->
9       <v-switch
10         v-model="isDark"
11         :label="isDark ? '다크 모드' : '라이트 모드'"
12         @change="toggleTheme"
13         inset
14         hide-details
15       ></v-switch>
16     </v-app-bar>
```

오늘의 핵심 요약



반응형 레이아웃 전략

CSS Grid의 유연함과 Bootstrap의 생산성을 비교하고, 상황에 맞는 레이아웃 설계 방법을 익혔습니다.



데이터 시각화

Chart.js와 vue-chartjs를 연동하여 Reactive한 데이터 바인딩 차트를 구현했습니다.



전문 UI 프레임워크

Vuetify를 활용해 완성도 높은 카드형 대시보드와 다크 모드 테마 시스템을 구축했습니다.



다음 단계

더 발전된 대시보드를 위한 제안

1

Grid 시스템 확장

1200px 이상 대화면에서 5열 이상 배치해보기

2

차트 고도화

Dataset 추가 및 톨팁/범례 커스터마이징

3

실전 기능 결합

Vuetify Data Table과 필터링 기능 연동하기

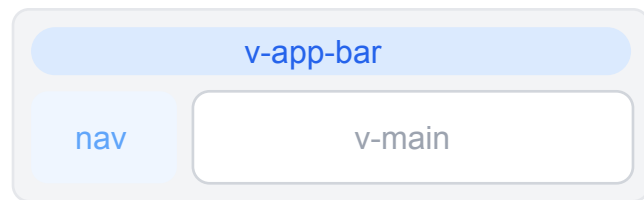
Vuetify 대표 컴포넌트 예시

<https://vuetifyjs.com/>

Layout & Grid

`<v-container>`, `<v-row>`

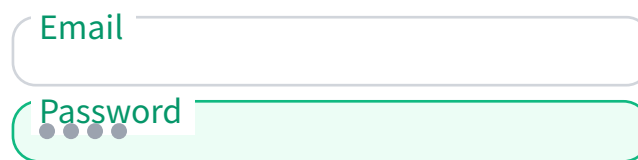
반응형 웹 앱의 기본 골격을 잡는 12컬럼 그리드 시스템. 자동 간격 조정(gap)과 정렬 기능 제공.



Form Inputs

`<v-text-field>`, `<v-select>`

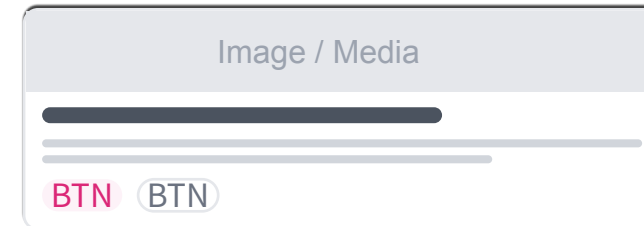
Material Design 스타일의 입력 폼. 유효성 검사 (Validation) 규칙 내장.



Cards & List

`<v-card>`, `<v-list>`

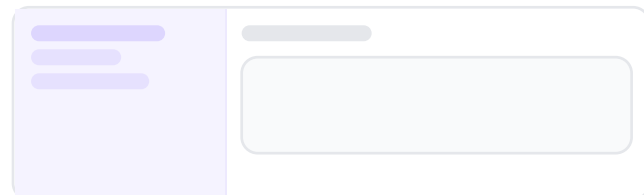
정보를 그룹화하여 보여주는 카드 UI. 이미지, 텍스트, 버튼 조합 가능.



Navigation

`<v-navigation-drawer>`

사이드바 메뉴, 앱바, 탭 등 사용자의 이동 경로를 제공하는 핵심 컴포넌트.



Data Table

`<v-data-table>`

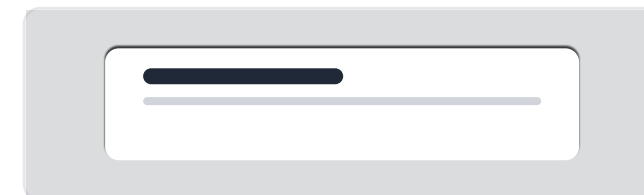
정렬, 필터, 페이지네이션이 포함된 강력한 테이블. 대시보드의 필수 요소.

Name	Status
Project A	Active
Project B	Pending
1-2 of 10	

Feedback

`<v-dialog>`, `<v-snackbar>`

모달 창(Dialog), 알림 메시지(Snackbar) 등 사용자와의 상호작용 UI.



대안 UI 프레임워크 소개

Vuetify 외에도 프로젝트 성격에 따라 선택할 수 있는 강력한 Vue 3 UI 라이브러리들이 있습니다.

Naive UI

Vue 3 전용

TypeScript

설정이 간편하고 디자인이 매우 깔끔합니다.
커스터마이징이 자유롭고 기본적으로 다크모드를 완벽 지원합니다.

트리 셰이킹(Tree Shaking) 지원

가볍고 빠른 성능

Element Plus

엔터프라이즈

대시보드

중국 알리바바(Alibaba) 계열에서 시작된
Element UI의 Vue 3 버전입니다.
복잡한 데이터 테이블과 폼 처리에 강력합니다.

방대한 컴포넌트 생태계

관리자(Admin) 페이지 표준

Quasar

All-in-One

Cross Platform

단일 코드베이스로 웹(SPA/SSR), 모바일 앱,
데스크탑 앱(Electron)까지 한 번에 빌드할 수
있습니다.

Material Design 2.0 준수

강력한 CLI 도구 제공

Tip: 단순한 반응형 웹사이트라면 Naive UI, 복잡한 백오피스라면 Element Plus, 앱까지 확장할 계획이라면 Quasar를 추천합니다.

APPENDIX



실무 문제 해결과 심화 가이드

수업 중 다루지 못한 상세한 원리와
실무에서 자주 마주치는 이슈들에 대한
해결책을 정리했습니다.



주요 내용

Key Topics



Chart.js 데이터 감지 문제 해결

참조 변경과 불변 업데이트(Immutable Update) 패턴



JavaScript 스프레드 연산자(...)

객체와 배열을 다루는 모던 자바스크립트 문법 심화



안전한 상태 관리 패턴

Vue와 React 등 모던 프레임워크 공통 베스트 프랙티스



이 자료들은 나중에 실무 프로젝트를 진행할 때
"왜 코드가 안 돌지?" 싶을 때 다시 찾아보면 큰 도움이 됩니다.

데이터가 바뀌어도 차트가 안 변한다?



문제 상황

버튼을 클릭해서 `updateData()`를 호출했지만,
콘솔에 데이터 변경은 찍히는데 **차트는 그대로인 현상**

```
// ❌ 동작하지 않는 코드 (내부 값만 변경)  
chartData.value.datasets[0].data = [...]
```

💡 객체의 참조(Reference)는 그대로 유지됨



vue-chartjs의 시선

"어? `props.data`의 주소값이 똑같네?
변경된 게 없으니 다시 그리지 않아도 되겠군."



해결 방법

불변 업데이트(Immutable Update) 패턴을 사용하여
객체 자체를 **새로운 객체로 교체**해야 합니다.

```
// ✅ 해결 코드 (새 객체 할당)  
chartData.value = { ...old, datasets: [...] }
```

✅ 객체의 참조(Reference)가 변경됨 → 감지 성공!

💡 핵심 요약

- **Vue**: 내부 속성이 바뀌어도 반응형 시스템이 감지함
- **vue-chartjs**: `:data props`의 **참조가 바뀔 때** 확실하게 리렌더링됨
- ★ 따라서 "통째로 교체"하는 것이 가장 안전한 방법!

Immutable 업데이트 패턴 적용

안 먹히는 이유

객체 내부의 값만 변경하면, chartData 변수 자체의 참조(주소)는 변하지 않습니다. vue-chartjs는 주로 props.data의 참조 변경을 감지하여 차트를 다시 그립니다.

해결: 통째로 교체

기존 객체를 수정하는 대신, 새로운 객체를 만들어 할당합니다. 이를 불변 업데이트(Immutable Update)라고 하며, 참조가 바뀌므로 확실하게 감지됩니다.

Key Point

"내부 데이터만 썩 바꾸지 말고, 새 그릇(객체)에 담아서 줘라!"

BarChart.vue (updateData 함수 수정)

```
1 // ❌ [BAD] 내부 데이터만 수정 (참조 유지 → 감지 불가)
2 function updateData(){
3   chartData.value.datasets[0].data=[...]/ * 생략 */
4 }

5 // ✅ [GOOD] 불변 업데이트 (새 객체로 교체 → 감지 성공)
6 function updateData(){
7   const old=chartData.value

8   // 1. 기존 설정 복사 + data만 새 배열로 교체한 '새 데이터셋' 생성
9   const newDataset={
10     ...old.datasets[0],
11     data:old.datasets[0].data.map(v⇒v+Math.round(Math.random()*100))
12   }

13   // 2. chartData 전체를 '새 객체'로 교체
14   chartData.value={
15     ...old,
16     datasets:[newDataset]
17   }
18 }
```

스프레드 연산자... 한 번에 이해하기

spread_syntax.js

```
1// 1. 기본 객체
2const user = { name: '철수', age: 20 }
3
4// 2. 단순 복사 (펼쳐 넣기)
5const copy = { ...user }
6// 결과: { name: '철수', age: 20 }
7
8// 3. 복사 + 덮어쓰기 (순서 중요!)
9const updated = {
10  ...user, // 1) 기존 값 다 펼치고
11  age: 25 // 2) age만 25로 덮어쓰기
12}
13// 결과: { name: '철수', age: 25 }
```

기존 속성은 유지하고, 변경할 속성만 뒤에 적습니다.



복사 후 덮어쓰기 패턴

Copy & Overwrite

스프레드 연산자(...)는 객체나 배열의 껍데기를 벗겨 내부 요소를 펼쳐서 (Spread) 복사합니다.

이후에 오는 프로퍼티는 기존 값을 덮어쓰기(Overwrite) 하므로, "기존 값 유지 + 일부 변경"에 최적화된 문법입니다.

우리 프로젝트에서의 활용

1

newDataset 생성

기존 데이터셋의 스타일(색상, 라벨 등)은 그대로 복사(...old.datasets[0])하고, data 값만 새 배열로 교체합니다.

2

chartData 업데이트

기존 차트의 라벨(labels) 등은 유지(...old)하고, datasets 배열만 통째로 교체하여 참조 변경을 일으킵니다.

실무 팀: 배열 스프레드와 안전한 업데이트

1 배열 스프레드 기본 문법

array-spread.js

```
const arr = [1, 2, 3]
const copy = [...arr] // [1, 2, 3] (복사본)
const withZero = [0, ...arr] // [0, 1, 2, 3] (앞에 추가)
const withFour = [...arr, 4] // [1, 2, 3, 4] (뒤에 추가)
```

2 VUE 안전한 업데이트 패턴

safe-update.js

```
// ❌ Bad: 내부 배열만 수정 (참조 동일)
chartData.value.datasets[0].data = [...newData]
```

```
// ✅ Good: 전체 객체 교체 (불변 업데이트)
chartData.value = {
  ...chartData.value, // 기존 속성 유지
  datasets: [
    {
      ...chartData.value.datasets[0],
      // data 배열도 새로 생성해서 할당
      data: [...chartData.value.datasets[0].data, 99]
    }
  ]
}
```



불변성(Immutability)과 참조 변경

Reference Identity & Re-rendering

Vue와 같은 프레임워크나 외부 라이브러리(Chart.js 등)는 성능 최적화를 위해 "객체의 참조(Reference)가 바뀌었는가?"를 기준으로 업데이트 여부를 결정하는 경우가 많습니다.

따라서 내부 값만 바꾸기보다, ...(Spread Operator)를 사용해 새로운 객체/배열을 만들어 할당하는 것이 가장 안전하고 확실한 방법입니다.

핵심 포인트

참조 변경 보장

새로운 메모리 주소를 가진 객체로 교체함으로써 Watcher가 100% 변화를 감지하도록 유도합니다.

Best Practice

React, Redux, Vue 생태계 전반에서 권장되는 "불변 업데이트(Immutable Update)" 패턴입니다.